

MLPy Workshop

February 1, 2026

1 Week 3: Clustering

Student Name 1, Student Name 2

In this workshop, we will work through a set of problems on clustering, another canonical form of unsupervised learning. Clustering is an important tool that is used to discover homogeneous groups of data points within a heterogeneous population. It can be the main goal in some problems, while in others it may be used in EDA to understand the main types of behavior in the data or in feature engineering.

We will start by generating some artificial data, and then we will utilize clustering algorithms described in lectures and explore the impact of feature engineering on the solution. We will then attempt to find clusters in a gene expression dataset.

As usual, the worksheets will be completed in teams of 2-3, using **pair programming**, and we have provided cues to switch roles between driver and navigator. When completing worksheets:

- You will have tasks tagged by (CORE) and (EXTRA).
- Your primary aim is to complete the (CORE) components during the WS session, afterwards you can try to complete the (EXTRA) tasks for your self-learning process.
- Look for the as cue to switch roles between driver and navigator.

Instructions for submitting your workshops can be found at the end of worksheet. As a reminder, **you must submit a pdf of your notebook on Learn by 16:00 PM on the Friday** of the week the workshop was given.

As you work through the problems it will help to refer to your lecture notes (navigator). The exercises here are designed to reinforce the topics covered this week. Please discuss with the tutors if you get stuck, even early on!

1.1 Outline

1. Problem Definition and Setup: Simulated Example
2. K-means: Simulated Example
3. Hierarchical Clustering: Simulated Example
4. Gene Expression Data
5. Hierarchical Clustering: Gene Expression Data
6. K-means Clustering: Gene Expression Data

2 Problem Definition and Setup: Simulated Example

2.1 Packages

First, lets load in some packages to get us started.

```
[1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster import hierarchy
```

2.2 Data: Simulated Example

We will begin with a simple simulated example in which there are truly three clusters. We assume that there are $D = 2$ features and within each cluster, the data points are generated from a spherical normal distribution $N(\mathbf{m}_k, \sigma_k^2 \mathbf{I})$ for clusters $k = 1, 2, 3$, where both the mean $\mathbf{m}_k$ and variance σ_k^2 are different across clusters. Specifically, we assume that:

- Cluster 1: contains $|C_1| = 500$ points with mean vector $\mathbf{m}_1 = \begin{pmatrix} 0 \\ 4 \end{pmatrix}$ with standard deviation $\sigma_1 = 2$.
- Cluster 2: contains $|C_2| = 250$ points with mean vector $\mathbf{m}_2 = \begin{pmatrix} 0 \\ -4 \end{pmatrix}$ with standard deviation $\sigma_2 = 1$.
- Cluster 3: contains $|C_3| = 100$ points with mean vector $\mathbf{m}_3 = \begin{pmatrix} -4 \\ 0 \end{pmatrix}$ with standard deviation $\sigma_3 = 0.5$.

Run the following code to generate the dataset described above.

```
[2]: # Number of features
D = 2

# Cluster sizes
N_1 = 500
N_2 = 250
N_3 = 100

# Cluster means
m_1 = np.array([0., 4.])
m_2 = np.array([0., -4.])
m_3 = np.array([-4., 0.])
m = np.vstack((m_1, m_2, m_3))

# Cluster standard deviations
sd_1 = 2.
```

```

sd_2 = 1.
sd_3 = 0.5

# Generate the data
rnd = np.random.RandomState(5)
X_1 = rnd.normal(loc = m_1, scale = sd_1, size = (N_1,D))
X_2 = rnd.normal(loc = m_2, scale = sd_2, size = (N_2,D))
X_3 = rnd.normal(loc = m_3, scale = sd_3, size = (N_3,D))
X = np.vstack((X_1, X_2, X_3))

# Save true cluster labels
cl = np.hstack((np.repeat(1,N_1),np.repeat(2,N_2),np.repeat(3,N_3)))

```

```

[3]: # Check that the size is correct
print(X.shape)

```

(850, 2)

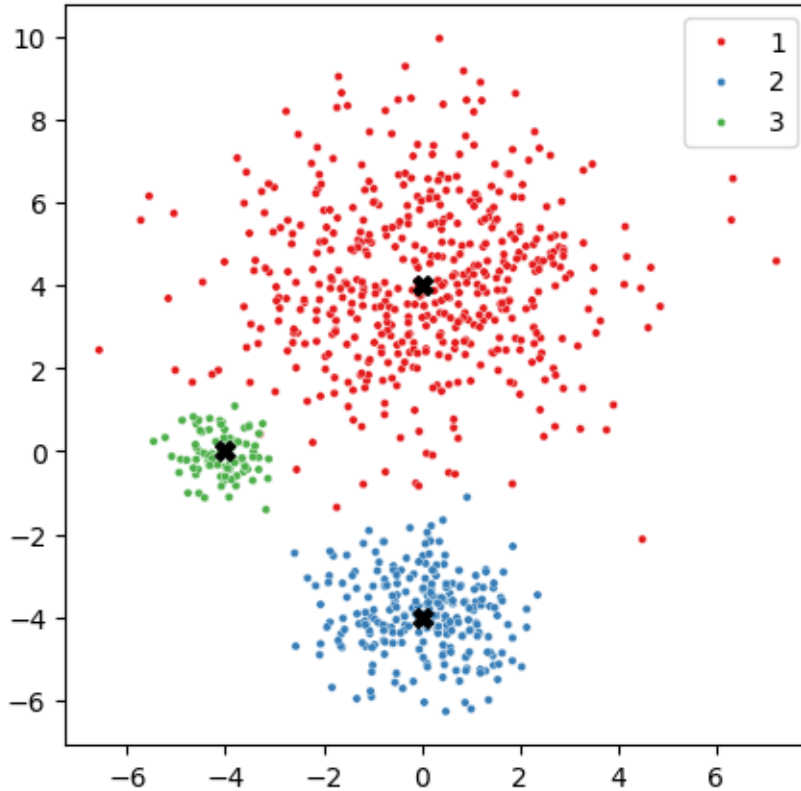
2.2.1 Exercise 1 (CORE)

Visualise the data and color by the true cluster labels.

```

[4]: # Plot the data and color by cluster membership
fig, ax = plt.subplots(1,1,figsize=(5,5))
sns.scatterplot(x=X[:,0], y=X[:,1], hue=cl, palette='Set1', s=10, ax=ax)
# Add cluster centers
ax.scatter(m[:,0], m[:,1], c='black', s=50, marker='X')
plt.show()

```



3 K-means Clustering: Simulated Example

To perform K-means clustering, we will use `KMeans()` in `sklearn.cluster`. Documentation is available [here](#), and for an overview of clustering methods available in `sklearn`, see [link](#). There are different inputs we can specify when calling `KMeans()` such as:

- `n_clusters`: the number of clusters.
- `init`: which specifies the initialization of the centroids, e.g. can be set to `k-means++` for K-means++ initialization or `random` for random initialization.
- `n_init`: which specifies the number of times the algorithm is run with different random initializations
- `random_state`: this can be set to a fixed number to make results reproducible.

We can then use the `.fit()` method of `KMeans` to run the K-means algorithm on our data.

After fitting, some of the relevant attributes of interest include:

- `labels_`: cluster assignments of the data points.
- `cluster_centers_`: mean corresponding to each cluster, stored in a matrix of size: number of clusters K times number features D .
- `inertia_`: the total within-cluster variation.

We can also call the methods `.transform()` and `.predict()` on our fitted `KMeans()` objects. The

former transforms/encodes an $N \times D$ feature matrix into an $N \times C$ matrix, where the new features represent the distance to cluster c , for $c = 1, \dots, C$. The latter predicts the cluster labels of each sample in an $N \times D$ feature matrix (i.e. returns the index of the closest cluster).

3.0.1 Exercise 2 (CORE)

Let's start by exploring how the clustering changes across the K-means iterations. To do, set:

- number of clusters to 3
 - initialization to random
 - number of times the algorithm is run to 1
 - fix the random seed to 0
- a) Now, fit the K-means algorithms with different values of the maximum number of iterations fixed to 1, 2, 3, and the default value of 300.
 - b) Plot the data points colored by cluster for the four different cases and mark the cluster centers to observe how the clustering solution changes across iterations.
 - c) How many iterations are needed for the convergence?

Hint

- To find the number of iterations, check the attributes of `KMeans`

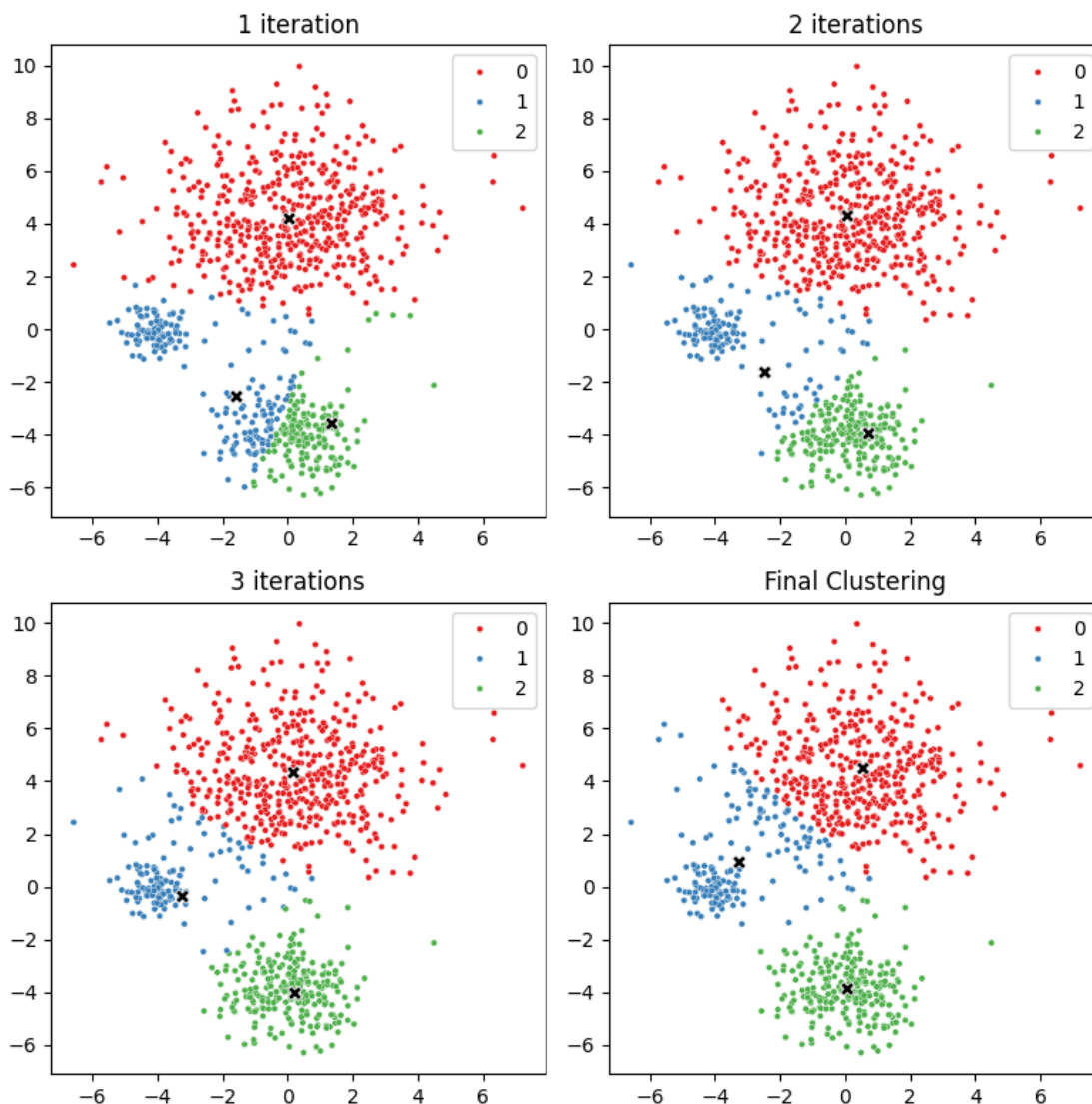
```
[5]: # Part a
kmeans_it1 = KMeans(n_clusters = 3, init = 'random', n_init = 1, random_state=0,
    ↪max_iter = 1).fit(X)
kmeans_it2 = KMeans(n_clusters = 3, init = 'random', n_init = 1, random_state=0,
    ↪max_iter = 2).fit(X)
kmeans_it3 = KMeans(n_clusters = 3, init = 'random', n_init = 1, random_state=0,
    ↪max_iter = 3).fit(X)
kmeans = KMeans(n_clusters = 3, init = 'random', n_init = 1, random_state=0).
    ↪fit(X)
```

```
[6]: # Part b
fig, ax = plt.subplots(2, 2, figsize=(8, 8))
ax = ax.flatten()
sns.scatterplot(x=X[:, 0], y=X[:, 1], hue=kmeans_it1.labels_, palette='Set1',
    ↪s=10, ax=ax[0])
sns.scatterplot(x=kmeans_it1.cluster_centers_[0], y=kmeans_it1.
    ↪cluster_centers_[1],
                c='black', s=50, marker='X', ax=ax[0])
ax[0].set_title('1 iteration')
sns.scatterplot(x=X[:, 0], y=X[:, 1], hue=kmeans_it2.labels_, palette='Set1',
    ↪s=10, ax=ax[1])
sns.scatterplot(x=kmeans_it2.cluster_centers_[0], y=kmeans_it2.
    ↪cluster_centers_[1],
                c='black', s=50, marker='X', ax=ax[1])
ax[1].set_title('2 iterations')
```

```

sns.scatterplot(x=X[:,0], y=X[:,1], hue=kmeans_it3.labels_, palette='Set1',
               ↪s=10, ax=ax[2])
sns.scatterplot(x=kmeans_it3.cluster_centers_[0], y=kmeans_it3.
               ↪cluster_centers_[1],
               c='black', s=50, marker='X', ax=ax[2])
ax[2].set_title('3 iterations')
sns.scatterplot(x=X[:,0], y=X[:,1], hue=kmeans.labels_, palette='Set1', s=10,
               ↪ax=ax[3])
sns.scatterplot(x=kmeans.cluster_centers_[0], y=kmeans.cluster_centers_[1],
               c='black', s=50, marker='X', ax=ax[3])
ax[3].set_title('Final Clustering')
fig.tight_layout()
plt.show()

```



Notice how the algorithm becomes better at discovering the well separated cluster centred at $(0, -4)^T$ as it progresses.

```
[7]: kmeans.n_iter_
```

```
[7]: 11
```

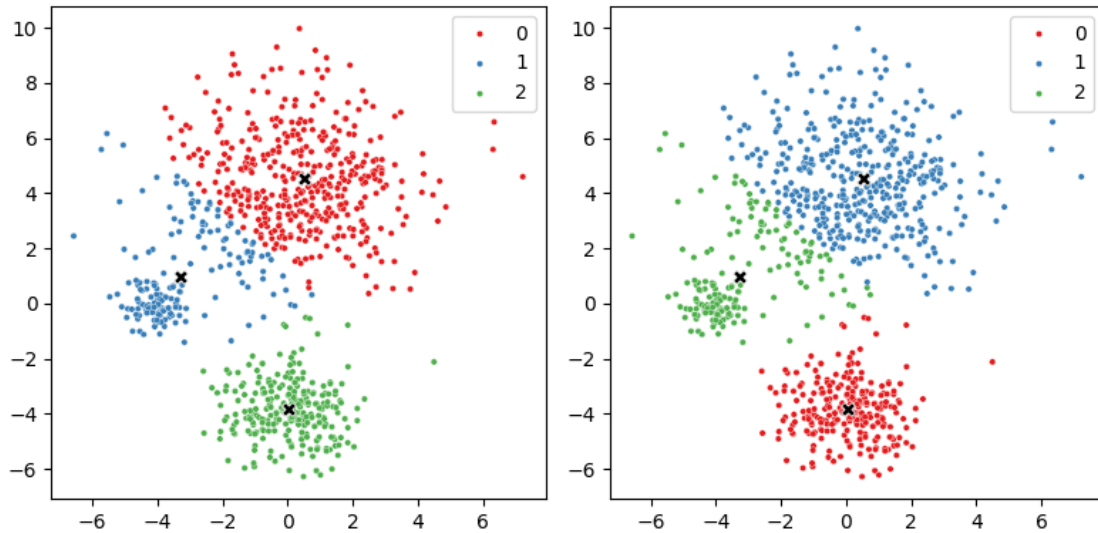
Only 11 iterations were required to reach convergence.

3.0.2 Exercise 3 (CORE)

Next, compare the random initialization with K-means++ (in this case fix the number of different initializations to 10). Plot both clustering solutions. Which requires fewer iterations? and which provides a lower within-cluster variation?

```
[8]: # Fit with both random and plus plus initializations
kmeans_rnd = KMeans(n_clusters = 3, init = 'random', n_init = 10, random_state=0).
    ↪ fit(X)
kmeans_pp = KMeans(n_clusters = 3, n_init = 10, random_state=0).fit(X)
```

```
[9]: # Plot both solutions
fig, ax = plt.subplots(1, 2, figsize=(8, 4))
sns.scatterplot(x=X[:, 0], y=X[:, 1], hue=kmeans_rnd.labels_, palette='Set1',
    ↪ s=10, ax=ax[0])
sns.scatterplot(x=kmeans_rnd.cluster_centers[:, 0], y=kmeans_rnd.
    ↪ cluster_centers[:, 1],
                c='black', s=50, marker='X', ax=ax[0])
sns.scatterplot(x=X[:, 0], y=X[:, 1], hue=kmeans_pp.labels_, palette='Set1',
    ↪ s=10, ax=ax[1])
sns.scatterplot(x=kmeans_pp.cluster_centers[:, 0], y=kmeans_pp.
    ↪ cluster_centers[:, 1],
                c='black', s=50, marker='X', ax=ax[1])
fig.tight_layout()
```



```
[10]: print('Number of iterations with random:', kmeans_rnd.n_iter_, ' and with ++:
      ↪', kmeans_pp.n_iter_)
```

Number of iterations with random: 11 and with ++: 4

```
[11]: print('Within-cluster variation with random:', kmeans_rnd.inertia_, ' and with ++:
      ↪', kmeans_pp.inertia_)
```

Within-cluster variation with random: 3833.3076162106936 and with ++: 3833.282646884114

In this case, the two algorithms find very similar clustering solutions. The ++ initialization requires fewer iterations (faster) and the solution has a slightly lower within-cluster variation.

3.0.3 Exercise 4 (CORE)

In the following two code cells, we compare the clustering solution using a different number of initializations equal to 1, 2, 5, 10, and 20 for kmeans++ initialization (first cell) and random initialization (second cell).

Based on the results, comment on the preferred initialization strategy and how many initializations are needed? Try changing the random state; how does that change your conclusions?

```
[12]: # Kmeans++ initialization
      # Plotting the clustering solution with different number of initializations
      n_init = np.array([1,2,5,10,20])

      rs = 0

      fig, ax = plt.subplots(1,n_init.shape[0],figsize=(20,4))
      for n in range(n_init.shape[0]):
```

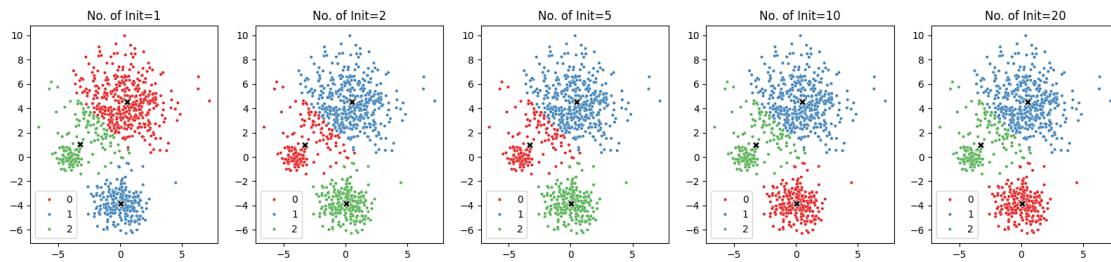


```

    kmeans_n = KMeans(n_clusters = 3, n_init = n_init[n], random_state=rs).
    ↪fit(X)
    sns.scatterplot(x=X[:,0], y=X[:,1], hue=kmeans_n.labels_, palette='Set1',
    ↪s=10, ax=ax[n])
    sns.scatterplot(x=kmeans_n.cluster_centers_[0], y=kmeans_n.
    ↪cluster_centers_[1],
                    c='black', s=50, marker='X', ax=ax[n])
    ax[n].set_title("No. of Init="+str(n_init[n]))
plt.show()

# Print the within cluster variation
for n in range(n_init.shape[0]):
    kmeans_n = KMeans(n_clusters = 3, n_init = n_init[n], random_state=rs).
    ↪fit(X)
    print("WCV="+str(round(kmeans_n.inertia_,4))+ ' for no. of init ='+
    ↪str(n_init[n]))

```



```

WCV=3833.368 for no. of init =1
WCV=3833.3076 for no. of init =2
WCV=3833.3076 for no. of init =5
WCV=3833.2826 for no. of init =10
WCV=3833.2826 for no. of init =20

```

```

[13]: # Random initialization
# Plotting the clustering solution with different number of initializations
n_init = np.array([1,2,5,10,20])

rs = 0

fig, ax = plt.subplots(1,n_init.shape[0],figsize=(20,4))
for n in range(n_init.shape[0]):
    kmeans_n = KMeans(n_clusters = 3, init = 'random', n_init = n_init[n],
    ↪random_state=rs).fit(X)
    sns.scatterplot(x=X[:,0], y=X[:,1], hue=kmeans_n.labels_, palette='Set1',
    ↪s=10, ax=ax[n])

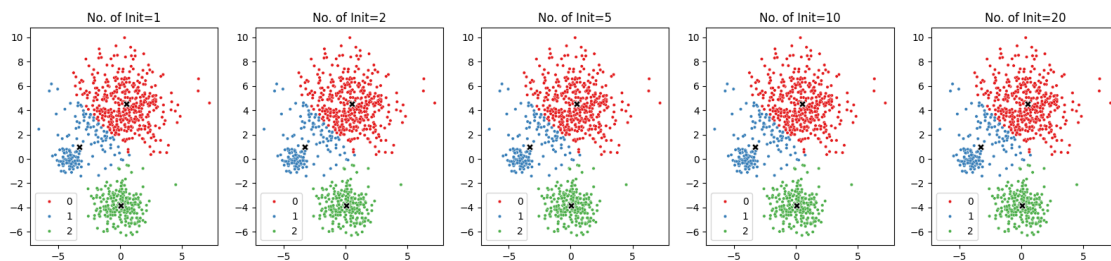
```

```

sns.scatterplot(x=kmeans_n.cluster_centers_[0], y=kmeans_n.
↪cluster_centers_[1],
                c='black', s=50, marker='X', ax=ax[n])
ax[n].set_title("No. of Init="+str(n_init[n]))
plt.show()

# Print the within cluster variation
for n in range(n_init.shape[0]):
    kmeans_n = KMeans(n_clusters = 3, init = 'random', n_init = n_init[n], ↪
↪random_state=rs).fit(X)
    print("WCV="+str(round(kmeans_n.inertia_,4))+ ' for no. of init =' + ↪
↪str(n_init[n]))

```



```

WCV=3833.3076 for no. of init =1
WCV=3833.3076 for no. of init =2
WCV=3833.3076 for no. of init =5
WCV=3833.3076 for no. of init =10
WCV=3833.3076 for no. of init =20

```

While all the clustering solutions are very similar, kmeans++ is the preferred solution, as it finds the solution with the lowest within-cluster variation. For kmeans++, the same solution is found with at least ten different initializations. However, if we change the random state, in some cases we find that we need more initializations. In general, it is recommended to run with a large number of initializations, such as 20 or even 50, to avoid undesirable local optima.

Now, is a good point to switch driver and navigator

3.0.4 Exercise 5 (CORE)

Since we simulated the data, we know the true number of clusters. However, in practice this number is rarely known. Find the K-means solution with different choices of K and plot the within-cluster variation as a function of K . What value(s) of K seem appropriate based on this plot?

```

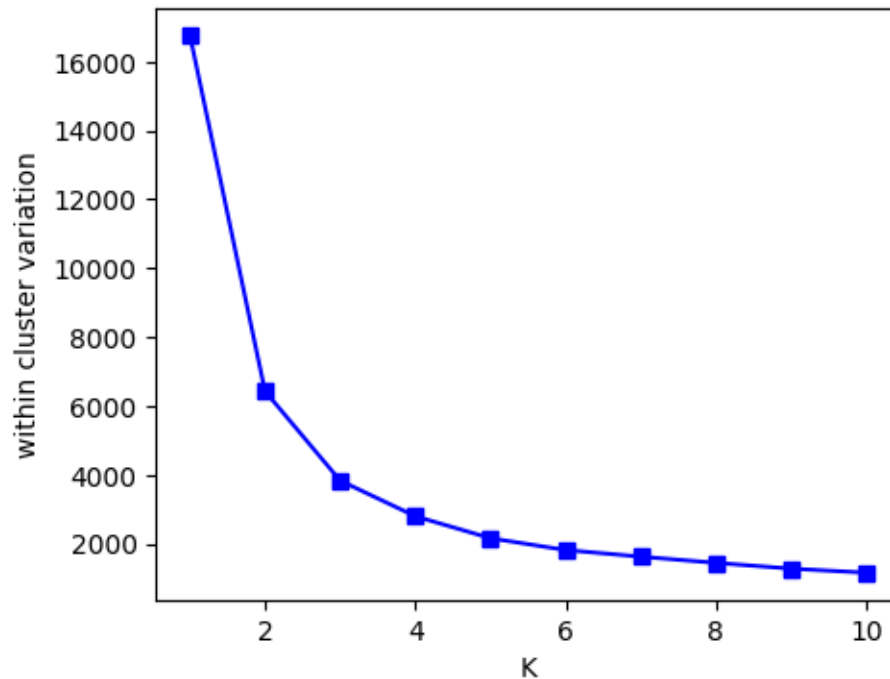
[14]: # Compute the within cluster variation for a range of choices of K
K = np.arange(1,11)
WCV = np.zeros(K.shape)

for i in range(K.shape[0]):

```

```
kmeans_K = KMeans(n_clusters = K[i], n_init = 20, random_state=0).fit(X)
WCV[i] = kmeans_K.inertia_
```

```
[15]: # Plot the within cluster variation
fig, ax = plt.subplots(1,1,figsize=(5,4))
ax.plot(K, WCV, c='b',marker='s')
plt.xlabel('K')
plt.ylabel('within cluster variation')
plt.show()
```



There appears to some leveling off after three clusters, which seems to be a good number of clusters.

3.0.5 Exercise 6 (CORE)

Now let's use the silhouette analysis to choose the number of clusters. In the following code cells, we use `silhouette_samples` to compute the silhouette coefficient for each data point, and `silhouette_score` which computes the mean of the silhouette coefficients across all data points.

- How many clusters would you choose based only on maximizing the silhouette score?
- Considering also the violin plots, which visualize the distribution of the silhouette coefficient across data points within each cluster, would you still choose the same number of clusters or pick a different number? Why?

```
[16]: from sklearn.metrics import silhouette_samples, silhouette_score
```

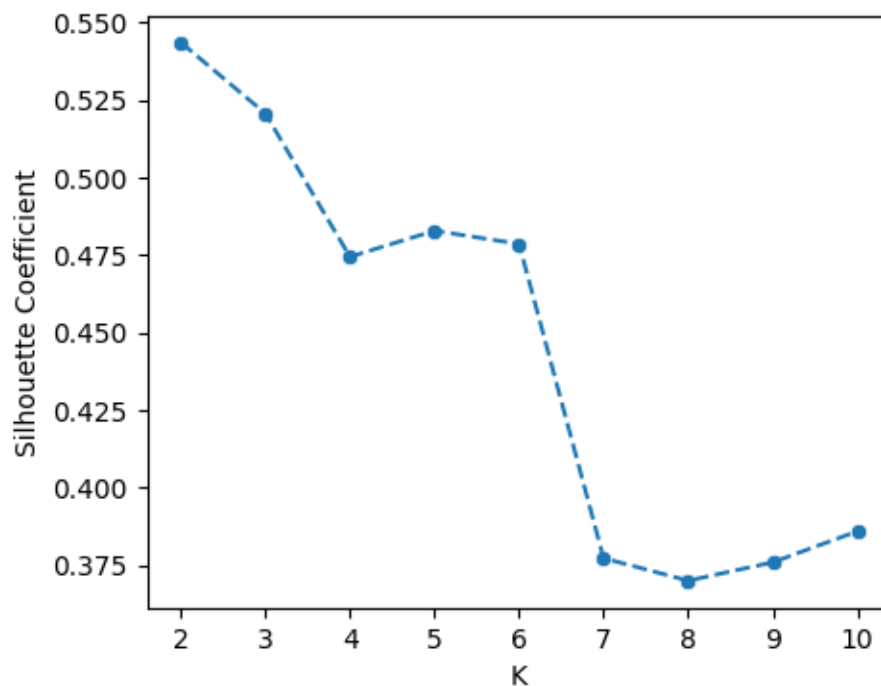
```

# Define the range of possible number of clusters
K = np.array([2, 3, 4, 5, 6, 7, 8, 9, 10])

# First plot the silhouette coefficient for different choices of K
silhouette_coeffs = np.zeros(K.shape)
for i in range(K.shape[0]):
    # Define kmeans object, fit, and predict labels
    kmeans_K = KMeans(n_clusters = K[i], n_init = 20, random_state=0).fit(X)
    labs = kmeans_K.predict(X)
    # Compute silhouette coefficient
    silhouette_coeffs[i] = silhouette_score(X, labs)

# Plot the silhouette coefficients
fig, ax = plt.subplots(1,1,figsize=(5,4))
sns.lineplot(x=K, y=silhouette_coeffs, linestyle='dashed', ax=ax)
sns.scatterplot(x=K, y=silhouette_coeffs, ax=ax)
plt.xlabel('K')
plt.ylabel('Silhouette Coefficient')
plt.show()

```



```

[17]: # Plot the silhouette samples for different choices of K

```

```

max_i = 4
fig, ax = plt.subplots(2, 5, figsize=(25, 10))

```

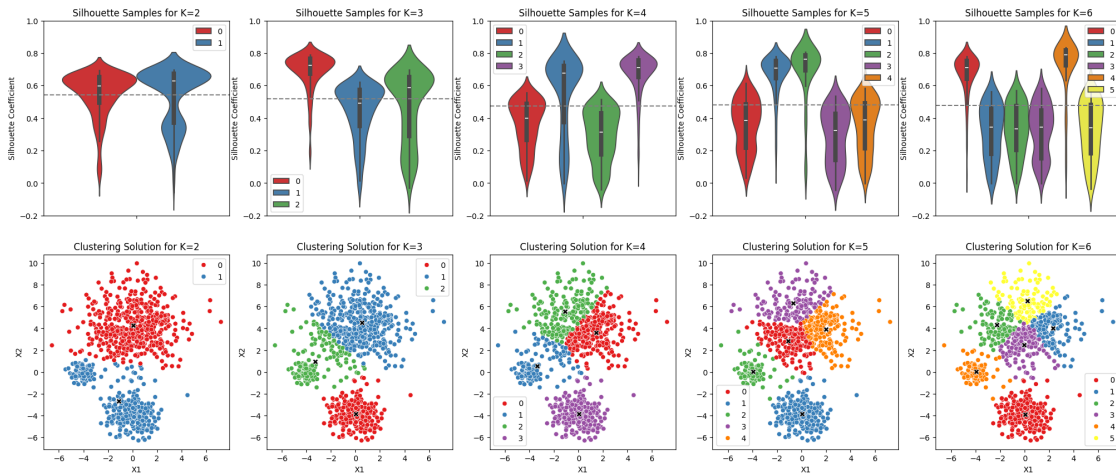
```

for i in range(max_i+1):
    kmeans_K = KMeans(n_clusters = K[i], n_init = 20, random_state=0).fit(X)
    labs = kmeans_K.predict(X)
    silhouette_samps = silhouette_samples(X, labs)

    # Create a violin plot of the silhouette samples
    sns.violinplot(y=silhouette_samps, hue=labs, ax=ax[0,i], palette='Set1')
    ax[0,i].axhline(silhouette_samps.mean(), ls='--',c="gray")
    ax[0,i].set_title(f'Silhouette Samples for K={K[i]}')
    ax[0,i].set_ylabel('Silhouette Coefficient')
    ax[0,i].set_xlabel('')
    ax[0,i].set_ylim([-0.2, 1])

    # Plot the clustering solution
    sns.scatterplot(x=X[:,0], y=X[:,1], hue=labs, ax=ax[1,i], palette='Set1')
    sns.scatterplot(x=kmeans_K.cluster_centers_[0], y=kmeans_K.
    ↪cluster_centers_[1],
                    c='black', s=50, marker='X', ax=ax[1,i])
    ax[1,i].set_title(f'Clustering Solution for K={K[i]}')
    ax[1,i].set_ylabel('X2')
    ax[1,i].set_xlabel('X1')

```



- If we want to maximize the average silhouette coefficient, we would choose 2 clusters.
- Here, may argue that three cluster solution is appropriate, even though average silhouette coefficient is slightly lower than the two cluster solution. In the three cluster solution, the red cluster of data points is well separated (the highest silhouette coefficients concentrating around 0.75). The green and blue clusters both have long tails, corresponding to some points with zero or negative coefficients (overlapping points between the clusters), but the majority of the points have coefficients above the average, suggesting a reasonable separation between the groups.

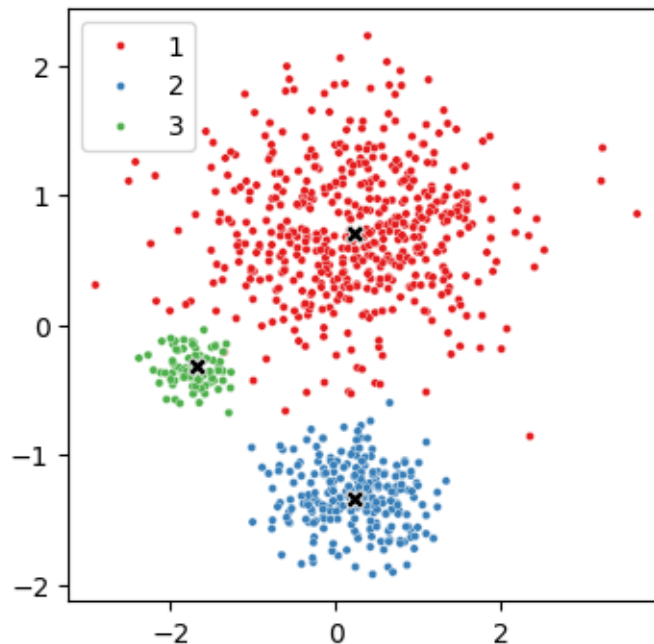
3.0.6 Exercise 7 (CORE)

Now standardize the data and re-run the K-means algorithm. Qualitatively, how has standardising the data impacted performance? Can you argue why you observe what you see?

```
[18]: from sklearn.preprocessing import StandardScaler

# Standardize the data
ss = StandardScaler()
Xs = ss.fit_transform(X)
ms = ss.transform(m)

# Plot the standardized data
fig, ax = plt.subplots(1,1,figsize=(4,4))
sns.scatterplot(x=Xs[:,0], y=Xs[:,1], hue=cl, ax=ax, palette='Set1', s=10)
sns.scatterplot(x=ms[:,0], y=ms[:,1], c='black', s=50, marker='X', ax=ax)
plt.show()
```



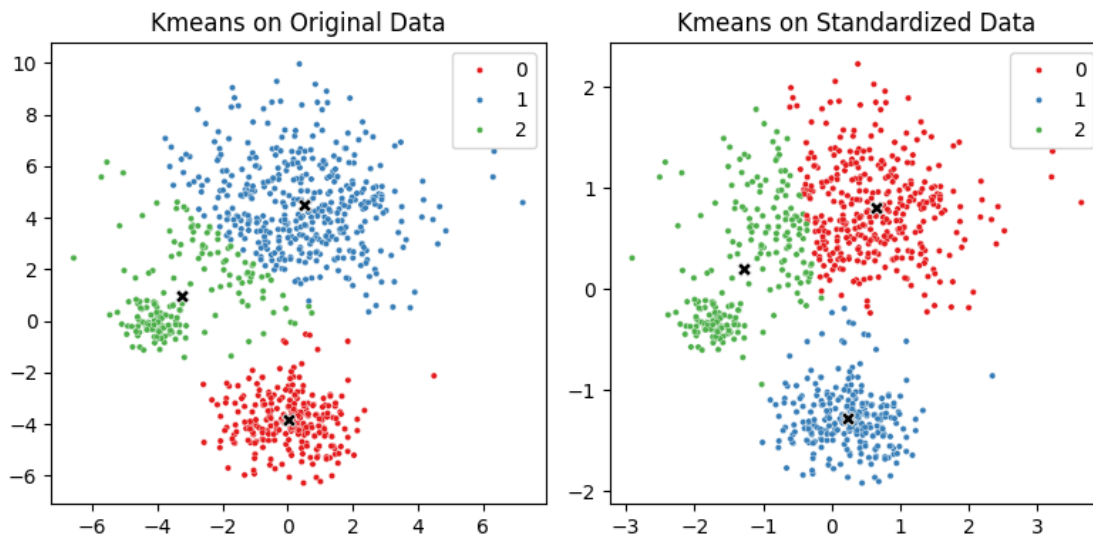
```
[19]: # Fit kmeans on the original data
kmeans_nos = KMeans(n_clusters = 3, n_init = 20, random_state=0).fit(X)
# Fit kmeans on the standardized data
kmeans_s = KMeans(n_clusters = 3, n_init = 20, random_state=0).fit(Xs)

fig, ax = plt.subplots(1,2,figsize=(8,4))
sns.scatterplot(x=X[:,0], y=X[:,1], hue=kmeans_nos.labels_, palette='Set1', s=10, ax=ax[0])
# sns.scatterplot(x=Xs[:,0], y=Xs[:,1], hue=kmeans_s.labels_, palette='Set1', s=10, ax=ax[1])
```

```

sns.scatterplot(x=kmeans_nos.cluster_centers[:,0], y=kmeans_nos.
    ↪cluster_centers[:,1],
                c='black', s=50, marker='X', ax=ax[0])
ax[0].set_title('Kmeans on Original Data')
sns.scatterplot(x=Xs[:,0], y=Xs[:,1], hue=kmeans_s.labels_, palette='Set1',
    ↪s=10, ax=ax[1])
sns.scatterplot(x=kmeans_s.cluster_centers[:,0], y=kmeans_s.cluster_centers[:,
    ↪1],
                c='black', s=50, marker='X', ax=ax[1])
ax[1].set_title('Kmeans on Standardized Data')
fig.tight_layout()

```



When we standardize the data, more of the large cluster 1 is grouped with the smaller cluster 3. As we discussed in lectures, standardizing the features is equivalent to weighting features in the dissimilarity by $1/\text{Var}(X_d)$. Thus, when standardizing, the influence of the second feature is reduced by approx $1/4$ relative to the first feature. As the second feature is important in distinguishing the smallest cluster 3 from the other two, this cluster gets obscured when standardizing.

```

[20]: # Compute the standard deviation of each feature
      np.std(X,axis=0)**2

```

```

[20]: array([ 4.4290757 , 15.31392339])

```

4 Hierarchical Clustering: Simulated Example

To perform hierarchical clustering, we will use the `linkage()` function from `scipy.cluster.hierarchy`. The inputs to specify include

- the data.

- **metric**: specifies the dissimilarity between data points. Defaults to the Euclidean distance.
- **method**: specifies the type of linkage, e.g. complete, single, or average.

Then, we can use `dendrogram()` from `scipy.cluster.hierarchy` to plot the dendrogram.

Note that you can also use `AgglomerativeClustering` from `sklearn.cluster`, which similarly has options for **metric** to specify the distance and **linkage** to specify the type of linkage. However, `sklearn` does not have its own functions for plotting the dendrogram and use must use the tools from `scipy.cluster.hierarchy`.

4.0.1 Exercise 8 (CORE)

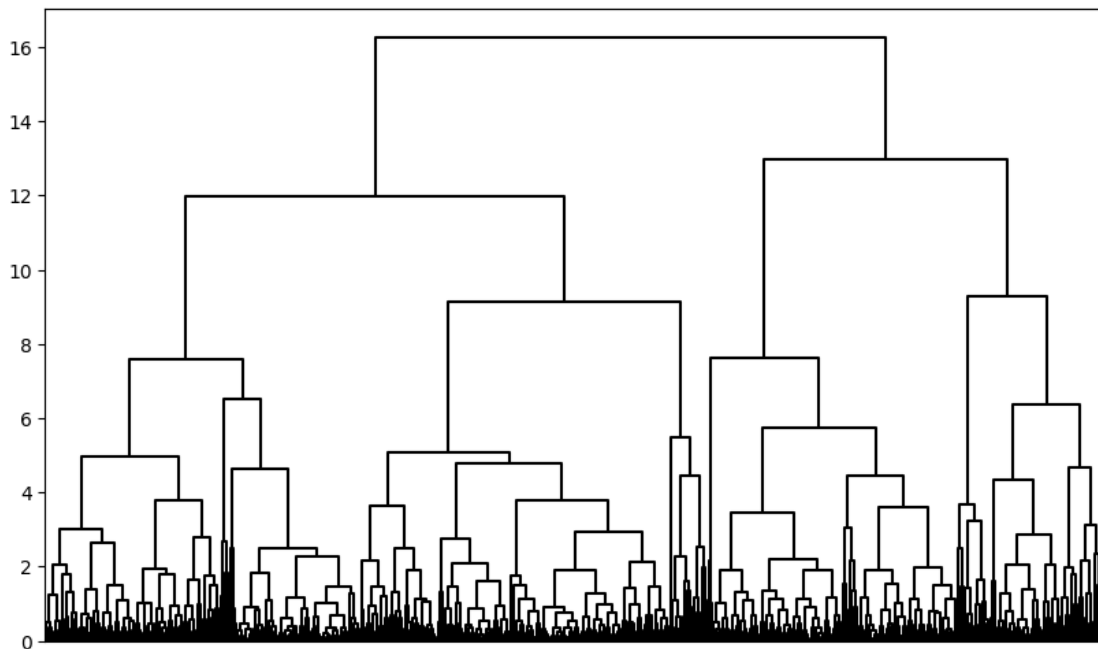
a) For the following code cell, what is dissimilarity and linkage is used in hierarchical clustering?

```
[21]: hc_comp = hierarchy.linkage(X, method='complete')
```

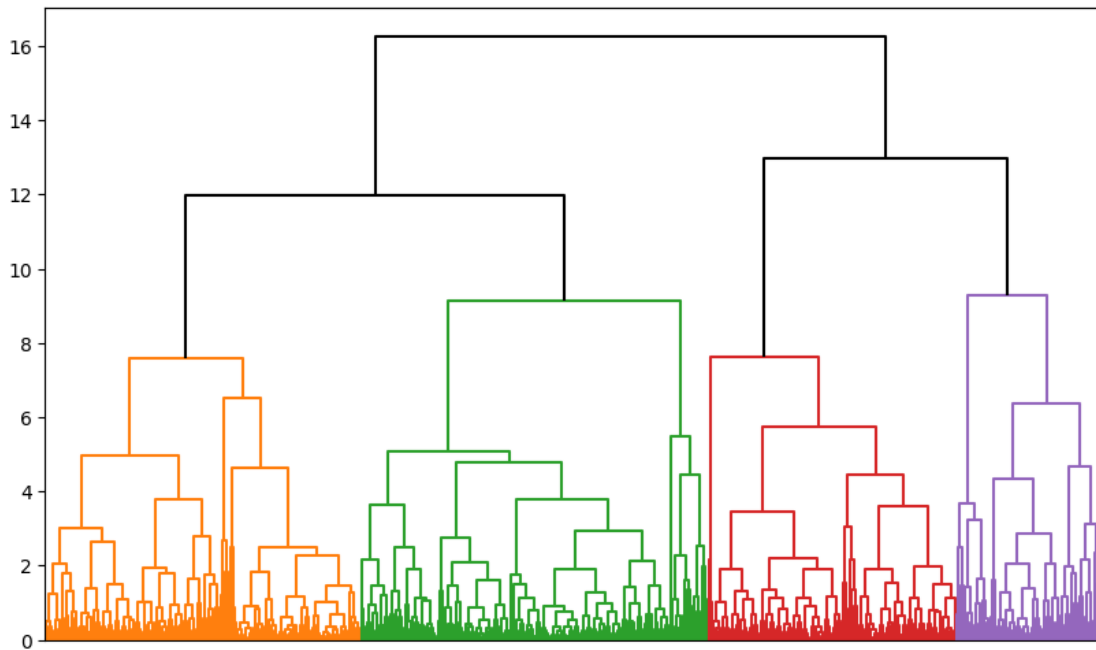
The code clusters the simulated data using hierarchical clustering with complete linkage and the Euclidean distance (default).

b) Plot the dendrogram by running the code below. Try changing the ‘color_threshold’ to a number (e.g. 11) to color the branches of the tree below the threshold with different colors. How many clusters are identified if the tree is cut at that threshold?

```
[22]: # Plot the dendrogram
fig, ax = plt.subplots(1, 1, figsize=(10,6))
hierarchy.dendrogram(hc_comp, ax=ax, no_labels=True,
                      color_threshold=np.inf, above_threshold_color='black')
plt.show()
```




```
[23]: # Change the color threshold to visualize clusters obtained by cutting the tree
fig, ax = plt.subplots(1, 1, figsize=(10,6))
hierarchy.dendrogram(hc_comp, ax=ax, no_labels=True,
                      color_threshold=11, above_threshold_color='black')
plt.show()
```



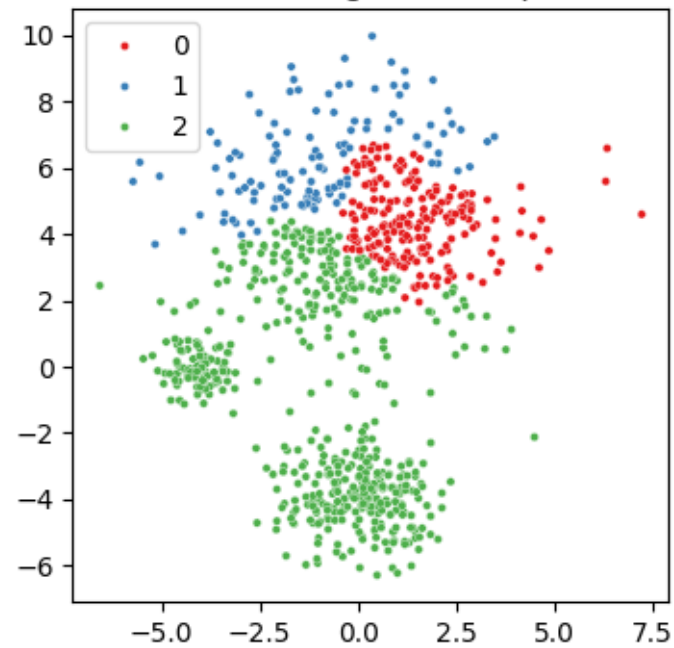
Answers will vary. If you choose a threshold of 11, 4 clusters are identified.

- c) Now, use the function `cut_tree()` from `scipy.cluster.hierarchy` to determine the cluster labels associated with a given cut of the dendrogram. You can either specify the number of clusters via `n_clusters` or the height/threshold at which to cut via `height`. Plot the data colored by the cluster labels.

```
[24]: # Cut the tree at a specified number of clusters
hc_cl = hierarchy.cut_tree(hc_comp, n_clusters=3)

fig, ax = plt.subplots(1,1,figsize=(4,4))
sns.scatterplot(x=X[:,0], y=X[:,1], hue=hc_cl.reshape(-1), palette='Set1',
               s=10, ax=ax)
ax.set_title('Hierarchical Clustering with complete linkage')
plt.show()
```

Hierarchical Clustering with complete linkage

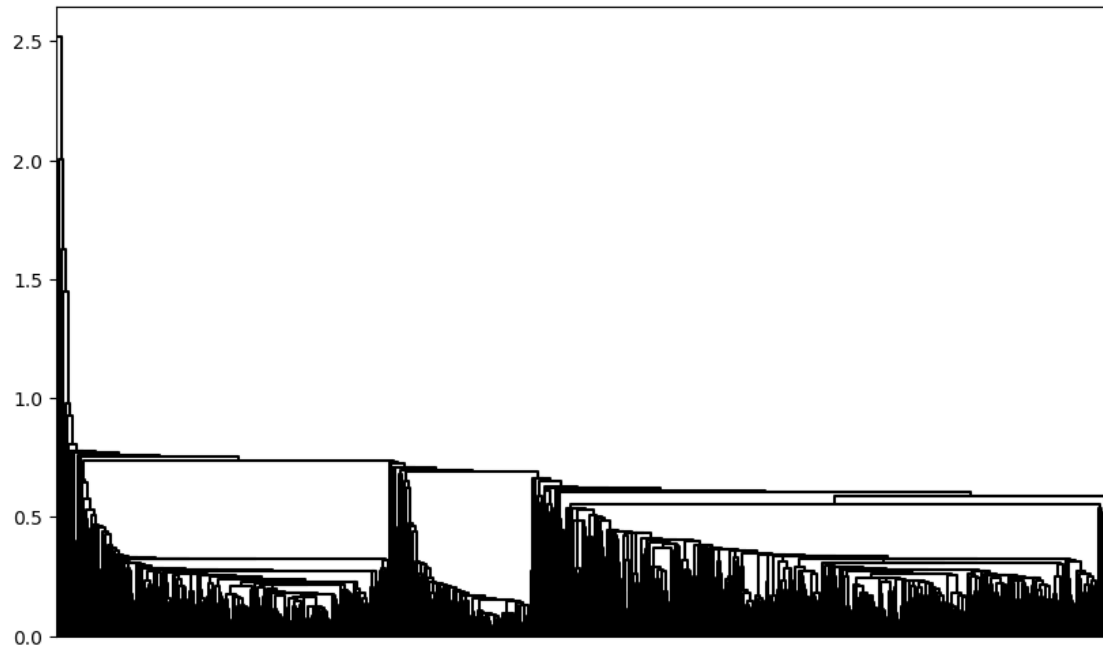


4.0.2 Exercise 9 (CORE)

Now try changing the linkage to single and average. Does this affect on the results?

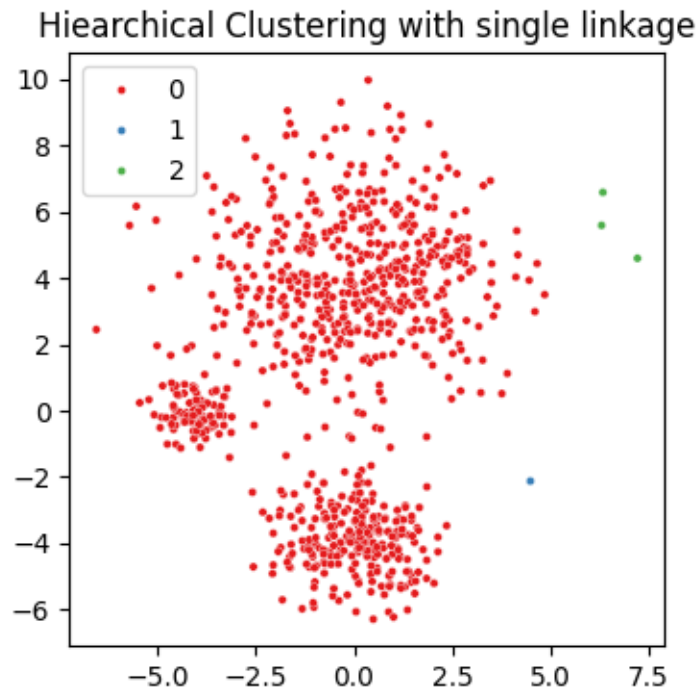
```
[25]: # Change to single linkage and plot the dendrogram
      hc_sing = hierarchy.linkage(X, method='single')

      # Plot the dendrogram
      fig, ax = plt.subplots(1, 1, figsize=(10,6))
      hierarchy.dendrogram(hc_sing, ax=ax, no_labels=True,
                           color_threshold=-np.inf, above_threshold_color='black')
      plt.show()
```



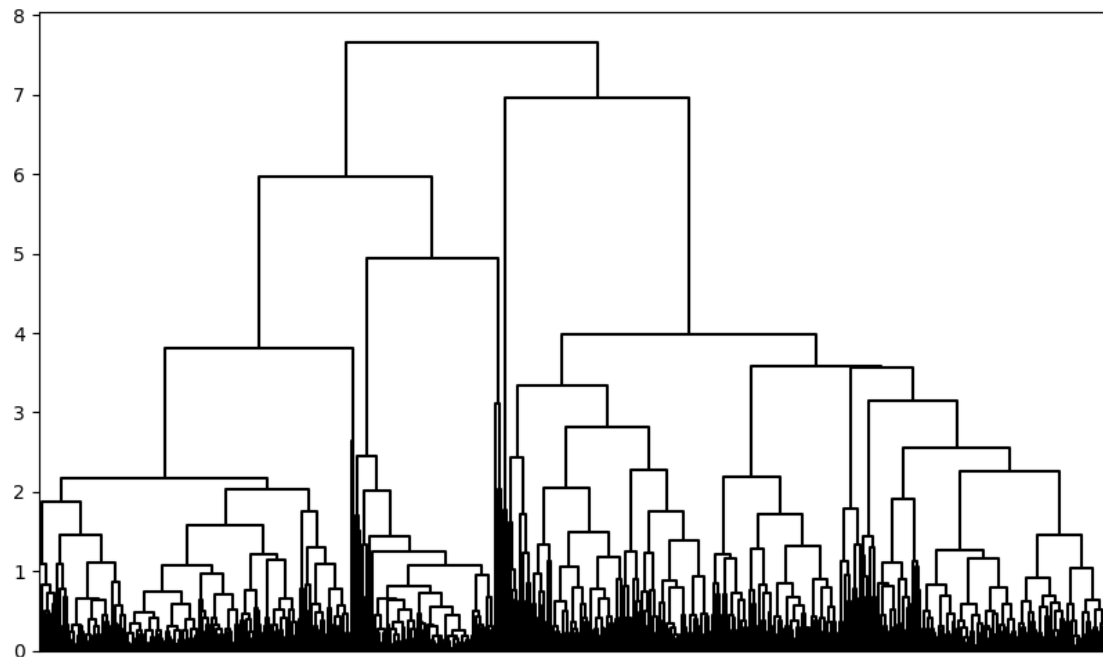
```
[26]: # Visualize the clustering solution obtained by cutting the tree
hc_cl = hierarchy.cut_tree(hc_sing, n_clusters=3)

fig, ax = plt.subplots(1,1,figsize=(4,4))
sns.scatterplot(x=X[:,0], y=X[:,1], hue=hc_cl.reshape(-1), palette='Set1',
               s=10, ax=ax)
ax.set_title('Hiarchical Clustering with single linkage')
plt.show()
```



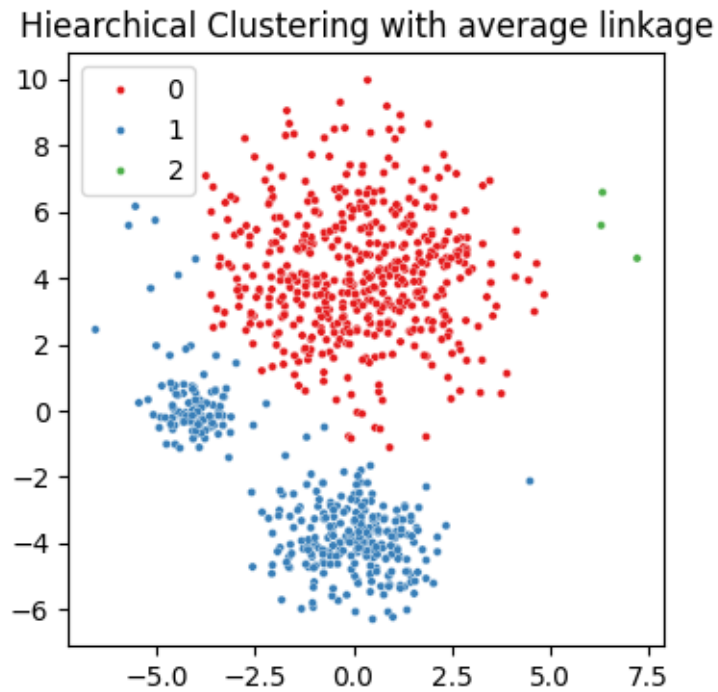
```
[27]: # Change to average linkage and plot the dendrogram
hc_avg = hierarchy.linkage(X, method='average')

# Plot the dendrogram
fig, ax = plt.subplots(1, 1, figsize=(10,6))
hierarchy.dendrogram(hc_avg, ax=ax, no_labels=True,
                     color_threshold=-np.inf, above_threshold_color='black')
plt.show()
```



```
[28]: # Visualize the clustering solution obtained by cutting the tree
hc_cl = hierarchy.cut_tree(hc_avg, n_clusters=3)

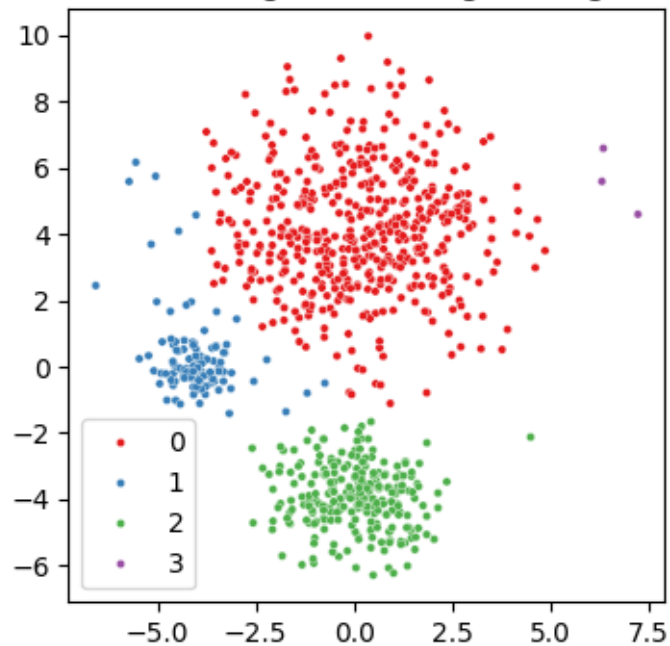
fig, ax = plt.subplots(1,1,figsize=(4,4))
sns.scatterplot(x=X[:,0], y=X[:,1], hue=hc_cl.reshape(-1), palette='Set1',
               s=10, ax=ax)
ax.set_title('Hiarchical Clustering with average linkage')
plt.show()
```



```
[29]: # Visualize the clustering solution obtained by cutting the tree
hc_cl = hierarchy.cut_tree(hc_avg, n_clusters=4)

fig, ax = plt.subplots(1,1,figsize=(4,4))
sns.scatterplot(x=X[:,0], y=X[:,1], hue=hc_cl.reshape(-1), palette='Set1',
               ↪s=10, ax=ax)
ax.set_title('Hierarchical Clustering with average linkage, 4 clusters')
plt.show()
```

Hierarchical Clustering with average linkage, 4 clusters



Yes, this has a great effect on the results. The results with single linkage do not recover the clusters well, and insteads tends to split off outlying points. Instead, average linkage with 4 clusters more closely resembles the true clustering in this example, with an additional cluster of some outlying points.

Now, is a good point to switch driver and navigator

5 Gene Expression Data

Now, we will consider a more complex real dataset with a larger feature space.

The dataset is the **NCI cancer microarray dataset** discussed in both *Introduction to Statistical Learning* and *Elements of Statistical Learning*. The dataset consists of $D = 6830$ gene expression measurements for each of $N = 64$ cancer cell lines. The aim is to determine whether there are groups among the cell lines with similar gene expression patterns. This is an example of a high-dimensional dataset with D much larger than N , which makes visualization difficult. The $N = 64$ cancer cell lines have been obtained from samples of cancerous tissues, corresponding to 14 different types of cancer. However, our focus remains unsupervised learning and we will use the cancer labels only to plot.

We first need to read in the dataset.

```
[30]: #Fetch the data and cancer labels
url_data = 'https://web.stanford.edu/~hastie/ElemStatLearn/datasets/nci.data.'
         ↪CSV'
```

```
url_labels = 'https://web.stanford.edu/~hastie/ElemStatLearn/datasets/nci.label.
↳txt'

X = pd.read_csv(url_data)
y = pd.read_csv(url_labels, header=None)

# clean data by dropping identifier column and transpose so that features are
↳columns
X = X.drop(labels='Unnamed: 0', axis=1).T
```

```
[31]: X.shape
```

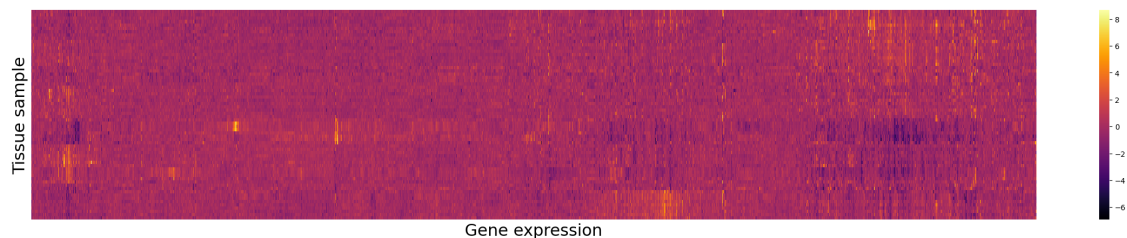
```
[31]: (64, 6830)
```

```
[32]: y.shape
```

```
[32]: (64, 1)
```

Let's visualise the data with a heatmap.

```
[33]: # Heatmap of the gene expression data
fig, ax = plt.subplots(1,1,figsize=(30,5))
sns.heatmap(X, cmap='inferno', ax=ax)
ax.set_xticks([])
ax.set_yticks([])
ax.set_xlabel("Gene expression", fontsize=22)
ax.set_ylabel("Tissue sample", fontsize=22)
plt.show()
```



We now convert our pandas dataframe into a numpy array and create integer labels for cancer type (for plotting purposes)

If you print the unique labels, you will notice there are lots of inconsistencies with white space etc. Run the following code to clean the labels.

```
[34]: # Print the unique labels and counts
y.value_counts()
```



```
[34]: 0
      OVARIAN          4
      RENAL            4
      MELANOMA         3
      COLON            3
      NSCLC            3
      BREAST           2
      CNS              2
      BREAST           2
      NSCLC            2
      NSCLC            2
      RENAL            2
      PROSTATE         2
      CNS              2
      MELANOMA         2
      LEUKEMIA         2
      COLON            2
      MELANOMA         2
      BREAST           1
      BREAST           1
      BREAST           1
      CNS              1
      LEUKEMIA         1
      COLON            1
      COLON            1
      MCF7D-repro      1
      LEUKEMIA         1
      LEUKEMIA         1
      MCF7A-repro      1
      LEUKEMIA         1
      K562A-repro      1
      K562B-repro      1
      NSCLC            1
      OVARIAN          1
      NSCLC            1
      MELANOMA         1
      OVARIAN          1
      RENAL            1
      RENAL            1
      RENAL            1
      UNKNOWN          1
      Name: count, dtype: int64
```

```
[35]: # Clean the labels by stripping the white space
      y_clean = np.asarray(y).flatten()
      for j in range(y_clean.size):
          y_clean[j] = y_clean[j].strip()
```

```
cancer_types = list(np.unique(y_clean))
cancer_groups = np.array([cancer_types.index(lab) for lab in y_clean])
```

```
[36]: pd.Series(y_clean).value_counts()
```

```
[36]: RENAL          9
      NSCLC          9
      MELANOMA       8
      BREAST         7
      COLON          7
      OVARIAN        6
      LEUKEMIA       6
      CNS            5
      PROSTATE       2
      UNKNOWN        1
      K562B-repro    1
      K562A-repro    1
      MCF7A-repro    1
      MCF7D-repro    1
      Name: count, dtype: int64
```

```
[37]: X_array = np.asarray(X)
```

5.0.1 Exercise 10 (EXTRA)

Perform a PCA of **X** to visualize the data. Plot the first few principal component scores and color by cancer type. Do cell lines within the same cancer types seems to have similar scores? Make a scree plot of the proportion of variance explained. How many components does this suggest?

```
[38]: from sklearn.decomposition import PCA

      # Fit PCA
      nci_pca = PCA()
      nci_scores = nci_pca.fit_transform(X_array)
```

```
[39]: # Plot the data in PC space
      figs, axes = plt.subplots(1,2, figsize=(15,6))

      # PC1 v PC2
      sns.scatterplot(x = nci_scores[:,0], y = nci_scores[:,1], hue = y_clean, s = 50, ax = axes[0])
      sns.move_legend(axes[0], "upper left", bbox_to_anchor=(1, 1))
      axes[0].set_xlabel('PC1')
      axes[0].set_ylabel('PC2')

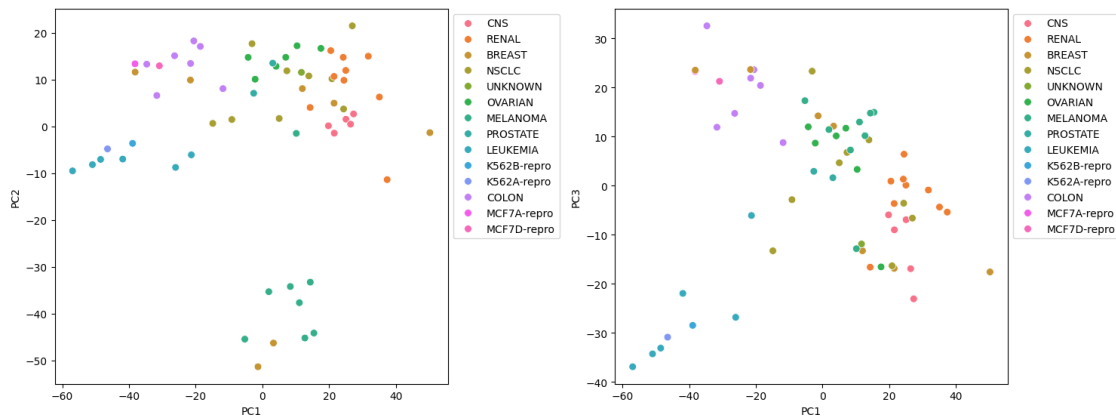
      # PC1 v PC3
```

```

sns.scatterplot(x = nci_scores[:,0], y = nci_scores[:,2], hue = y_clean, s = 50, ax = axes[1])
sns.move_legend(axes[1], "upper left", bbox_to_anchor=(1, 1))
axes[1].set_xlabel('PC1')
axes[1].set_ylabel('PC3')

figs.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

```

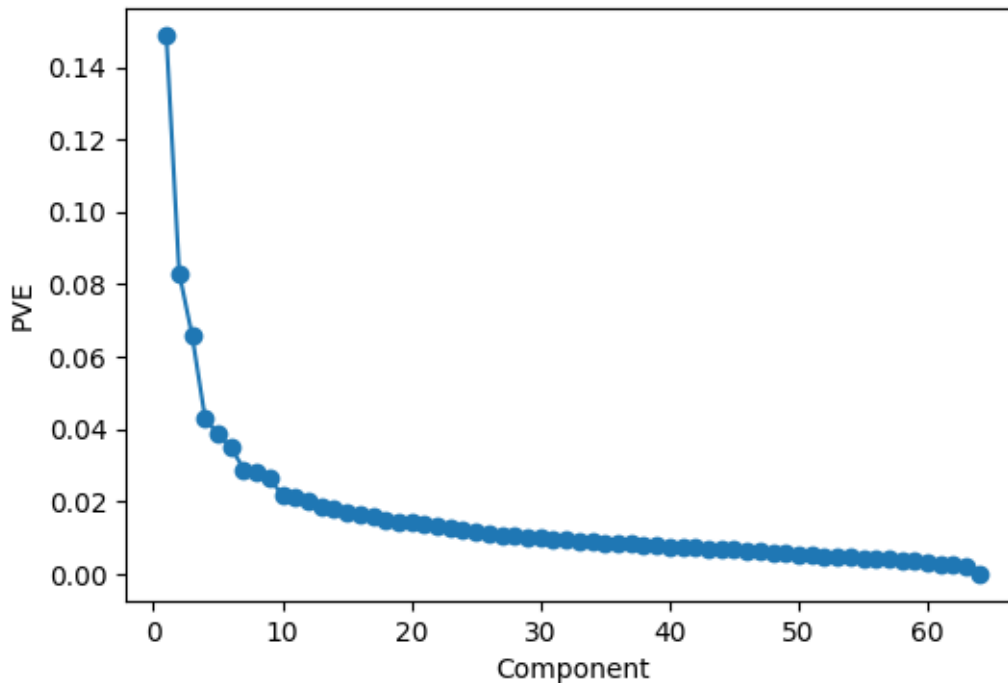


Yes, overall, cell lines within the same cancer have similar PC scores.

```

[40]: # Plot the proportion of variance explained
fig, ax = plt.subplots(1,1,figsize=(6,4))
ticks = np.arange(nci_pca.n_components_)+1
ax.plot(ticks,nci_pca.explained_variance_ratio_,marker='o')
ax.set_xlabel('Component')
ax.set_ylabel('PVE')
plt.show()

```



```
[41]: sum(nci_pca.explained_variance_ratio_[0:6])
```

```
[41]: 0.41436706962962266
```

There does seem to be an elbow around 7 components, which together explain approx 41% of the variance.

6 Hierarchical Clustering: Gene Expression Data

Now, let's perform hierarchical clustering on the gene expression data.

6.0.1 Exercise 11 (CORE)

- Plot the dendrogram with complete, single, and average linkage. Does the choice of linkage affect the results? Which linkage would you choose?

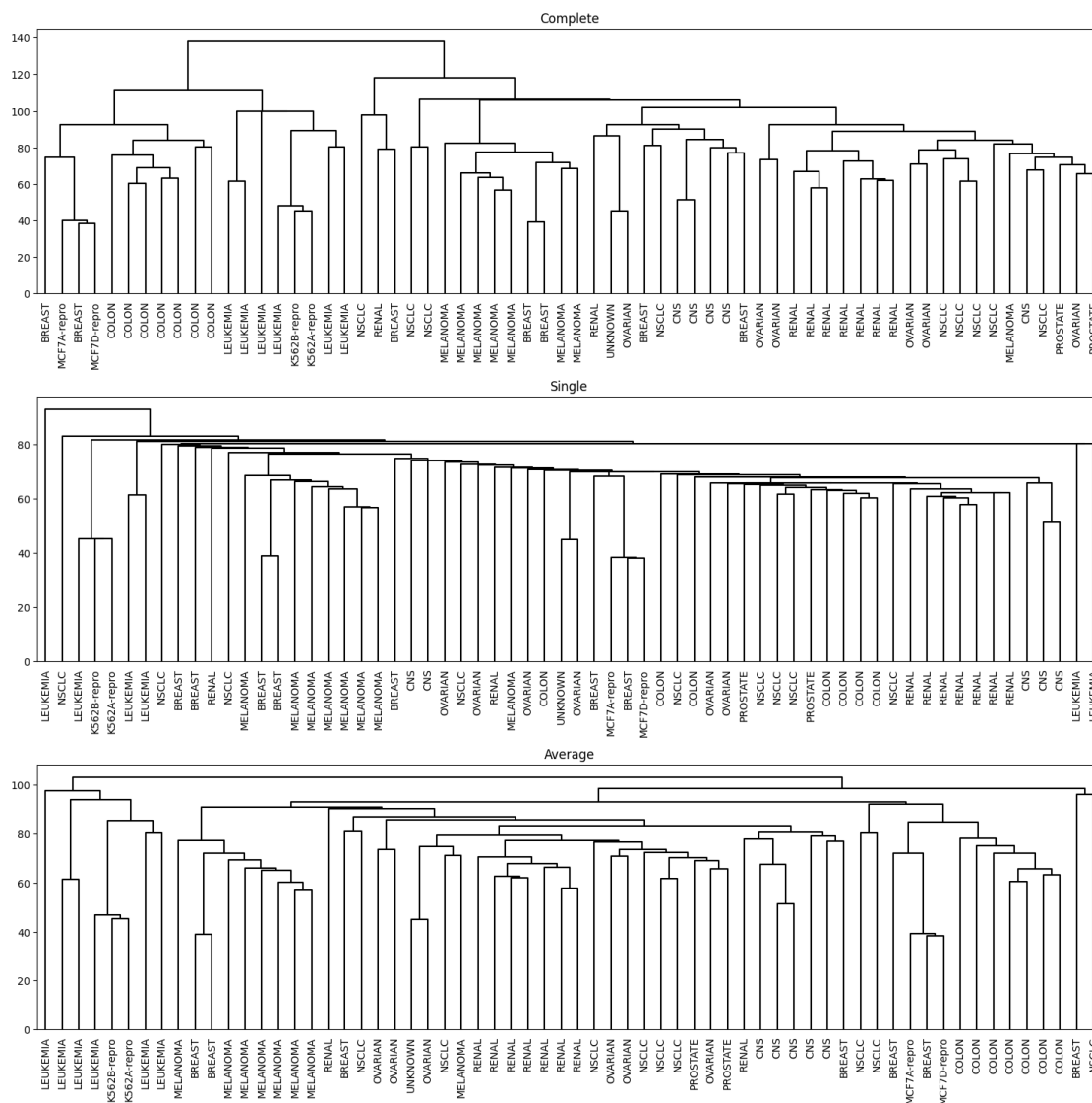
```
[42]: # Fit hierarchical clustering with different types of linkage
hc_comp = hierarchy.linkage(X, method='complete')
hc_sing = hierarchy.linkage(X, method='single')
hc_avg = hierarchy.linkage(X, method='average')

# Plot the dendrogram
cargs = {'color_threshold':-np.inf, 'above_threshold_color':'black',
        ↪ 'leaf_font_size':10}
fig, ax = plt.subplots(3,1, figsize=(15,15))
```

```

hierarchy.dendrogram(hc_comp, ax=ax[0], labels = np.asarray(y_clean), **cargs)
hierarchy.dendrogram(hc_sing, ax=ax[1], labels = np.asarray(y_clean), **cargs)
hierarchy.dendrogram(hc_avg, ax=ax[2], labels = np.asarray(y_clean), **cargs)
ax[0].set_title('Complete')
ax[1].set_title('Single')
ax[2].set_title('Average')
fig.tight_layout(pad = 1.1)
plt.show()

```

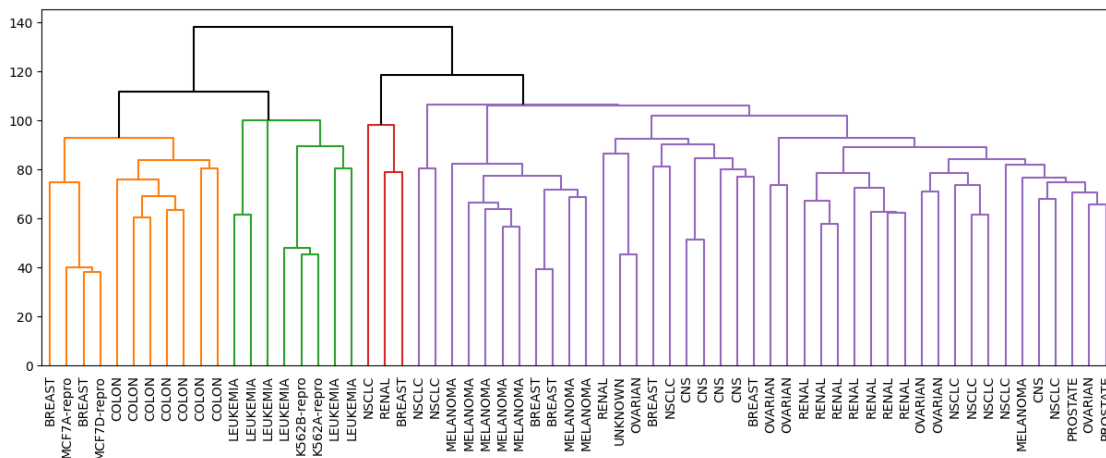


Single linkage tends to produce trailing clusters, that is, a very large cluster onto which single data points attach one-by-one. Complete and average produce more balanced clusters, and thus are generally preferred.

b) Select a linkage and a number of clusters (by examining the dendrogram and jumps in

the heights of the clusters merged). Plot the dendrogram and color the branches to identify the clusters. Use the option `labels = np.asarray(y_clean)`, `leaf_font_size=10` in `hierarchy.dendrogram` to add the cancer types as labels for each data point. Do you observe any patterns between the clusters and cancer types? You may also want to use `pd.crosstab` to compute a cross-tabulation to compare the clusters and cancer types.

```
[43]: # Plot the dendrogram with colors identifying clusters at a cut of the tree
fig, ax = plt.subplots(1, 1, figsize=(15,5))
hierarchy.dendrogram(hc_comp, ax=ax, labels = np.asarray(y_clean),
    ↳leaf_font_size=10,
                                color_threshold=110, above_threshold_color='black')
plt.show()
```



```
[44]: # Cross-tabulation of the cluster labels and cancer types
comp_cl = hierarchy.cut_tree(hc_comp, n_clusters=4)
pd.crosstab(pd.Series(y_clean, name='Cancer'), pd.Series(comp_cl.reshape(-1),
    ↳name = 'Hierarchial Clustering'))
```

```
[44]: Hierarchial Clustering  0  1  2  3
Cancer
BREAST                      4  1  0  2
CNS                         5  0  0  0
COLON                       0  0  0  7
K562A-repro                 0  0  1  0
K562B-repro                 0  0  1  0
LEUKEMIA                    0  0  6  0
MCF7A-repro                 0  0  0  1
MCF7D-repro                 0  0  0  1
MELANOMA                    8  0  0  0
NSCLC                       8  1  0  0
OVARIAN                     6  0  0  0
```

PROSTATE	2	0	0	0
RENAL	8	1	0	0
UNKNOWN	1	0	0	0

There are some clear patterns. All the leukemia cell lines belong to one cluster and all the colon cell lines belong to another cluster. However the breast cancer cell lines are spread across three of the different clusters.

Note: your answer and choice of linkage and number of clusters may vary.

Now, is a good point to switch driver and navigator

7 K-means Clustering: Gene Expression Data

Now, let's perform k-means clustering on the gene expression data.

7.0.1 Exercise 12 (CORE)

Perform K-means clustering with the same number of clusters that you selected for hierarchical clustering. Are the results similar?

```
[45]: # Fit kmeans
kmeans = KMeans(n_clusters = 4, n_init = 20, random_state=0).fit(X)

# Compare hierarchical clustering with kmeans
pd.crosstab(pd.Series(kmeans.labels_, name = 'K-means'), pd.Series(comp_cl.
↪ reshape(-1), name = 'Hierarchical Clustering'))
```

```
[45]: Hierarchical Clustering    0    1    2    3
K-means
0                9    0    0   11
1               24    3    0    0
2                0    0    8    0
3                9    0    0    0
```

First note, that the clusters labels are arbitrary and swapping the labels does not change the clustering solution. We find that one cluster is identical in both solutions (the cluster consisting of mostly leukemia cells). However, the first cluster in K-means groups together points from cluster 1 and 4 in hierarchical clustering.

7.0.2 Exercise 13 (EXTRA)

Plot the two clustering solutions along with a plot of the data colored by the cancer types in the space spanned by the first two principal components.

```
[46]: # Plot the data in PC space
figs, axes = plt.subplots(1,3, figsize=(20,6))

# Color by cancer types
```

```

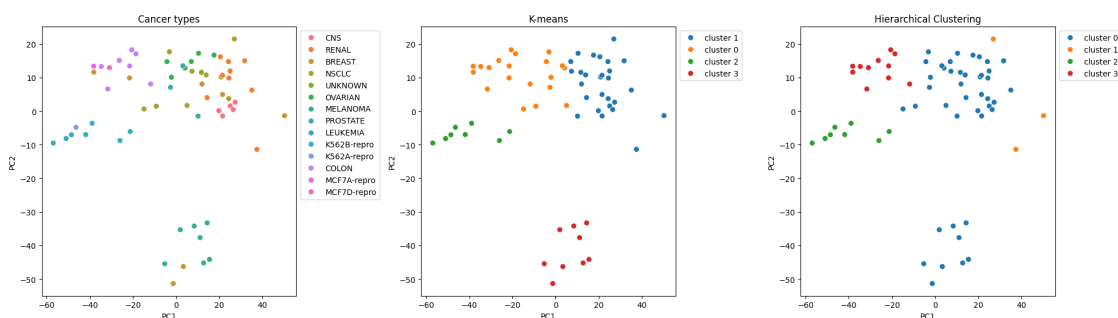
sns.scatterplot(x = nci_scores[:,0], y = nci_scores[:,1], hue = y_clean, s = 50, ax = axes[0])
sns.move_legend(axes[0], "upper left", bbox_to_anchor=(1, 1))
axes[0].set_xlabel('PC1')
axes[0].set_ylabel('PC2')
axes[0].set_title('Cancer types')

# Color by kmeans
text = "cluster "
kmeans_lbs = [f"{text}{l}" for l in kmeans.labels_]
sns.scatterplot(x = nci_scores[:,0], y = nci_scores[:,1], hue = kmeans_lbs, s = 50, ax = axes[1])
sns.move_legend(axes[1], "upper left", bbox_to_anchor=(1, 1))
axes[1].set_xlabel('PC1')
axes[1].set_ylabel('PC2')
axes[1].set_title('K-means')

# Color by hierarchical clustering
text = "cluster "
hc_lbs = [f"{text}{l}" for l in comp_cl.flatten()]
sns.scatterplot(x = nci_scores[:,0], y = nci_scores[:,1], hue = hc_lbs, s = 50, ax = axes[2])
sns.move_legend(axes[2], "upper left", bbox_to_anchor=(1, 1))
axes[2].set_xlabel('PC1')
axes[2].set_ylabel('PC2')
axes[2].set_title('Hierarchical Clustering')

figs.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

```



8 Competing the Worksheet

At this point you have hopefully been able to complete all the CORE exercises and attempted the EXTRA ones. Now is a good time to check the reproducibility of this document by restarting the notebook's kernel and rerunning all cells in order.

Before generating the PDF, please **change ‘Student 1’ and ‘Student 2’ at the top of the notebook to include your name(s).**

Once that is done and you are happy with everything, you can then run the following cell to generate your PDF.

```
[47]: # !jupyter nbconvert --to pdf mlp_week03_key.ipynb
```